

Dynamic Programming

Dynamic Programming

- An algorithm design technique (like divide and conquer)
- Divide and conquer
 - Partition the problem into independent subproblems
 - Solve the subproblems recursively
 - Combine the solutions to solve the original problem

Dynamic Programming

- Applicable when subproblems are **not** independent
 - Subproblems share subsubproblems

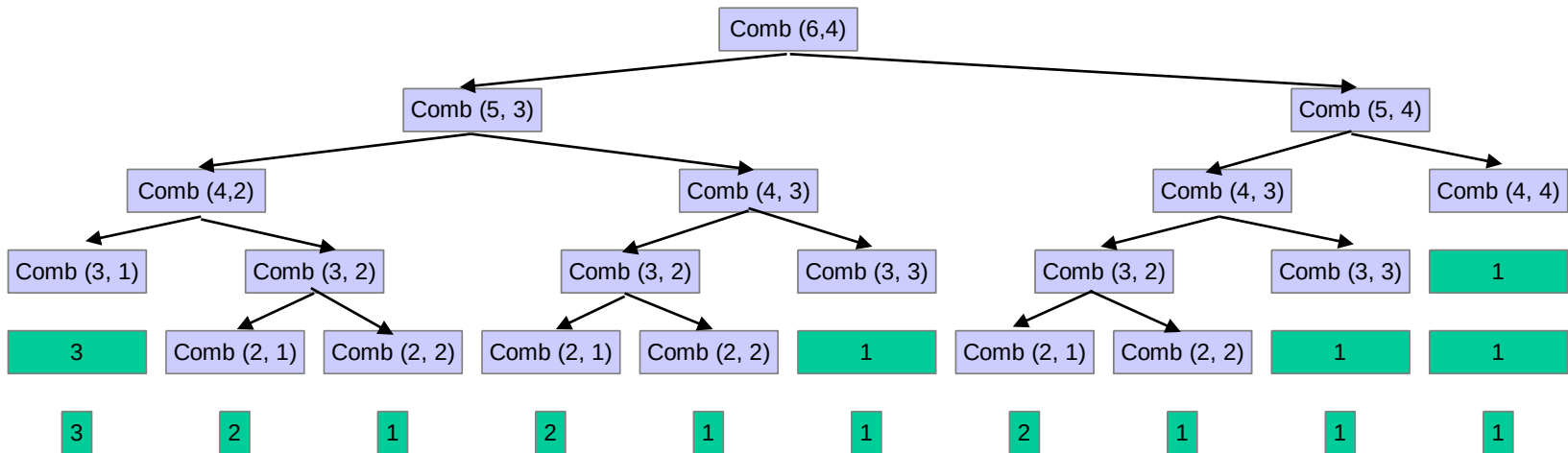
E.g.: Combinations:

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

$$\binom{n}{0} = 1 \qquad \binom{n}{n} = 1$$

- A divide and conquer approach would repeatedly solve the common subproblems
- Dynamic programming solves every subproblem just once and stores the answer in a table

Example: Combinations



$$\begin{pmatrix} n \\ k \end{pmatrix} = \begin{pmatrix} n-1 \\ k \end{pmatrix} + \begin{pmatrix} n-1 \\ k-1 \end{pmatrix}$$

Example 1: Fibonacci numbers

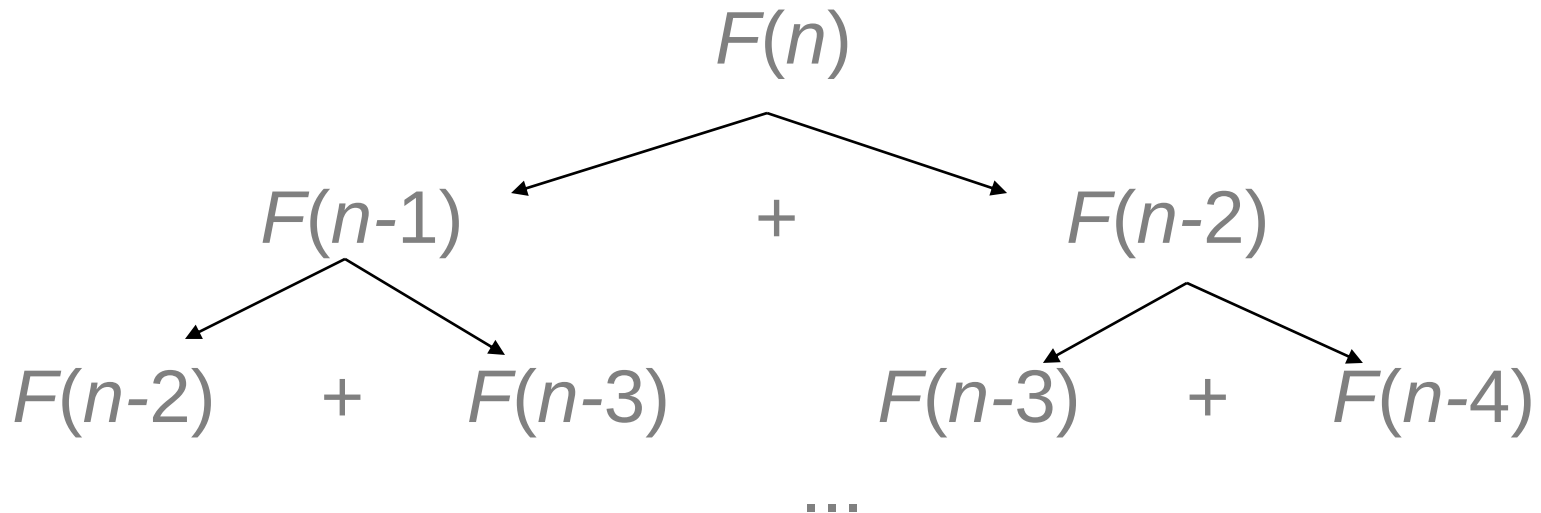
- Recall definition of Fibonacci numbers:

$$F(n) = F(n-1) + F(n-2)$$

$$F(0) = 0$$

$$F(1) = 1$$

- Computing the n^{th} Fibonacci number recursively (top-down):



Example 1: Fibonacci numbers

(cont.)

Computing the n^{th} Fibonacci number using bottom-up iteration and recording results:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(2) = 1 + 0 = 1$$

...

$$F(n-2) =$$

$$F(n-1) =$$

$$F(n) = F(n-1) + F(n-2)$$

0	1	1	. . .	$F(n-2)$	$F(n-1)$	$F(n)$
----------	----------	----------	--------------	----------------------------	----------------------------	--------------------------

Efficiency:

- time
- space

Example: Combinations

- ▶ If you want to make a 2-person committee from a group of four people. How many **different combinations** are possible?
- ▶ The number of ways to do this is given by $C(4, 2)$.

Consider 4 people A, B, C, D. Now the different combinations for making a committee of 2 persons from these 4 are;

A, B

A, C

A, D

C, D

B, D

B, C

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$C(4, 2) = \binom{4}{2} = \frac{4!}{2!(4-2)!} = 6$$

- ▶ Specifically, the **binomial coefficient** $C(n, k)$ counts the number of ways to form an **unordered collection of k items** chosen from a collection of **n distinct items**.

Example: Combinations

- ▶ The definition of binomial coefficient is given as:

$$\binom{n}{k} = \begin{cases} 1 & \text{if } k = 0 \text{ or } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{if } 0 < k < n \\ 0 & \text{Otherwise} \end{cases}$$

```
function C(n, k)
    if k=0 or k=n then return 1
    else return C(n-1, k-1) + C(n-1, k)
```


Example: Combinations

	0	1	2	3	4	5	..	k
0								
1		+						
2		+	+					
3		+	+	+				
4								
..								
n								

PASCAL's
TRIANGLE

Mx	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

```
function C(n, k)
  if k=0 or k=n then return 1
  else return C(n-1, k-1) + C(n-1, k)
```

Make Change Problem: Dynamic Programming solution

- ▶ We need to generate a table $c[n][N]$, where
 1. n = number of denominations
 2. N = amount for which you need to make a change.

To generate table $c[i][j]$ use following steps:

Step-1: Make $c[i][0] = 0$ for $0 < i \leq n$

Repeat step-2 to step-4 for the remaining matrix values

Step-2: If $i = 1$ then $c[i][j] = 1 + c[1][j - d_1]$

Step-3: If $j < d_i$ then $c[i][j] = c[i - 1][j]$

Step-4: Otherwise $c[i][j] = \min(c[i - 1][j], 1 + c[i][j - d_i])$

Optimal
Sub-structure

Make Change Problem: Dynamic Programming solution

► Denominations: $d_1 = 1, d_2 = 4, d_3 = 6$. Make a change of **Rs. 8**.

Step-1: Make $c[i][0] = 0$ for $0 < i \leq n$

Step-2: If $i = 1$ then $c[i][j] = 1 + c[1][j - d_1]$, here $d_1 = 1$

Step-3: If $j < d_i$ then $c[i][j] = c[i - 1][j]$

Step-4: Otherwise $c[i][j] = \min(c[i - 1][j], 1 + c[i][j - d_i])$

$j \longrightarrow$ Amount

		0	1	2	3	4	5	6	7	8
$i = 1$	$d_1 = 1$	0	1	2	3	4	5	6	7	8
$i = 2$	$d_2 = 4$	0	1	2	3	1	2			
$i = 3$	$d_3 = 6$	0								

Make Change Problem: Dynamic Programming solution

► We can also find the coins to be included in the solution set as follows:

1. Start looking at $c[3, 8] = c[2, 8] \Rightarrow$ So, **not to include** a coin with denomination 6.
2. Next go to $c[2, 8] \neq c[1, 8]$ but $c[2, 8] = 1 + c[2, 4]$ $c[i][j] = \min(c[i-1][j], 1 + c[i][j - d_i])$
 - So, include a coin with denomination 4
3. Now, got to $c[2, 4] \neq c[1, 4]$ but $c[2, 4] = 1 + c[2, 0]$
 - So, again include a coin with denomination 4
4. Go to $c[2, 0] = c[1, 0]$ and stop.

	0	1	2	3	4	5	6	7	8
$d_1 = 1$	0	1	2	3	4	5	6	7	8
$d_2 = 4$	0	1	2	3	①	2	3	4	②
$d_3 = 6$	0	1	2	3	1	2	1	2	2

Solution contains 2 coins with denomination 4

Dynamic Programming

- Used for **optimization problems**
 - A set of choices must be made to get an optimal solution
 - Find a solution with the optimal value (minimum or maximum)
 - There may be many solutions that lead to an optimal value
 - Our goal: **find an optimal solution**

Dynamic Programming Algorithm

1. **Characterize** the structure of an optimal solution
2. **Recursively** define the value of an optimal solution
3. **Compute** the value of an optimal solution in a bottom-up fashion
4. **Construct** an optimal solution from computed information (not always necessary)